

Robot Movement Planning

Constraint Programming

Alessio Russo (mat. 376856)

1 Problem Statement

We consider a stream of identical boxes arriving on a belt. Each box has size $x \times y$, where x is parallel to the belt direction and y is orthogonal to it. A target pallet capable of containing n boxes is given. Each box i has a destination position on the pallet, given by $(P_x[i], P_y[i])$, where $i = 1, \dots, n$. The robot can pick a consecutive set of boxes from the stream. Since boxes on the belt have no gaps between them, each picked set is contiguous along the x -axis. Given a gripper size S , the total length of the picked set must be at most $S + x$. This corresponds to an overflow of at most $x/2$ on each side of the gripper. Therefore, the maximum number of boxes that can be picked at once is $\left\lfloor \frac{S+x}{x} \right\rfloor$. After picking a set of boxes, the robot releases it on the pallet.

The gripper has two parallel plates, but only one plate can move during release. For this reason, the opening side must be free. It can face either the outside of the pallet or a position that has not been filled yet. We split the problem into two parts:

1. **Chunking.** Identify the sets of boxes that can be picked together. Each chunk must contain boxes on the same row, aligned along the x -axis, and must fit within the gripper capacity.
2. **Placement.** Choose the positions in which the chunks are released. For each chunk, also choose which plate opens. At every step, the selected opening side must be free.

The goal is to find a sequence of pick-and-release actions that places all boxes in their target positions while respecting the gripper capacity and avoiding collisions during release.

1.1 Parameters and Derived Constants

The proposed MiniZinc model uses five input parameters. The parameter `n_boxes` denotes the total number of boxes on the pallet. The parameter `boxes_x_dim` denotes the number of boxes in each row, and `pallet_rows` denotes the number of rows on the pallet. The parameter `boxes_x_dim` denotes the box dimension parallel to the gripper. Finally, `grip_size` denotes the gripper size S . These parameters are declared as follows:

```
1 par int: n_boxes;  
2 par int: boxes_x_dim;  
3 par int: boxes_per_row;  
4 par int: pallet_rows;  
5 par int: grip_size;
```

From these inputs, the model derives three constants. The first one is the maximum number of boxes that can be picked in a single operation, computed as:

$$\text{max_pkble_boxes} = \left\lfloor \frac{\text{grip_size} + \text{boxes_x_dim}}{\text{boxes_x_dim}} \right\rfloor.$$

This value derives directly from the gripper capacity $S + x$. The second constant is the number of chunks needed to cover one pallet row:

$$\text{chunks_per_row} = \left\lceil \frac{\text{boxes_per_row}}{\text{max_pkble_boxes}} \right\rceil.$$

This is a ceiling division because the last chunk in a row may contain fewer boxes than the maximum allowed number. The third constant is the total number of chunks:

$$\text{n_chunks} = \text{pallet_rows} \cdot \text{chunks_per_row}.$$

It is obtained by multiplying the number of chunks per row by the number of rows. The corresponding MiniZinc declarations are:

```

1 par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;
2
3 par int: chunks_per_row = (boxes_per_row + max_pkble_boxes - 1)
4                       div max_pkble_boxes;
5
6 par int: n_chunks = pallet_rows * chunks_per_row;
```

2 Chunking

This part of the model admits several possible encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each chunk explicitly stores the identifiers of all boxes it contains. Since chunks may have different lengths, this representation requires a two-dimensional array with padding values for shorter chunks.

```

1 array[1..n_chunks, 1..max_pkble_boxes] of var 0..n_boxes: chunks;
```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes chunk membership explicit and easy to inspect. However, it introduces a large number of decision variables and requires additional constraints to enforce the structure of a valid solution: each box must appear exactly once, and the non-empty positions inside each chunk must be ordered.

A second possibility is a *binary assignment* encoding, where a boolean variable states whether box i belongs to chunk g . This representation can be written as follows:

```

1 array[1..n_chunks, 1..n_boxes] of var bool: belongs_to_chunk;
```

In this case, chunk membership is represented by the truth value of `belongs_to_chunk[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation, and it does not encode contiguity directly: that property must be reconstructed through additional constraints.

For this reason, the adopted model represents each chunk using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each chunk is described by only two variables, and contiguity is implicit: once the start index and the length are known, the boxes in the chunk are uniquely determined. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid chunks.

```

1 array[1..n_chunks] of var 1..n_boxes: chunk_start;
2 array[1..n_chunks] of var 1..max_pkble_boxes: chunk_len;

```

The adopted encoding uses two arrays: `chunk_start`, which stores the first box of each chunk, and `chunk_len`, which stores the number of boxes in that chunk.

The following constraints define the position of each chunk in the input stream.

```

1 constraint chunk_start[1] = 1;

```

The first constraint sets the start of the first chunk to box 1. This fixes the beginning of the chunk sequence and ensures that the partition starts from the first box.

```

1 constraint forall(g in 1..n_chunks - 1)(
2   chunk_start[g + 1] = chunk_start[g] + chunk_len[g]
3 );

```

The second constraint makes the chunks consecutive. For each chunk g , the next chunk starts immediately after it, since the start position of chunk $g + 1$ is obtained by adding the length of chunk g to `chunk_start[g]`. This prevents gaps and overlaps between adjacent chunks.

```

1 constraint chunk_start[n_chunks] + chunk_len[n_chunks] - 1 = n_boxes;

```

The final constraint forces the last chunk to end exactly at box `n_boxes`. Together, these constraints ensure that the chunks form a complete partition of the input stream, where every box is covered exactly once.

3 Placement

Once the chunks have been defined, the model must decide where each chunk is placed on the pallet. More precisely, for each chunk it must choose the pallet row on which the chunk is dropped and the horizontal position at which the chunk starts. This is represented by two decision variables:

```

1 array[1..n_chunks] of var 1..boxes_along_y: y_row;
2 array[1..n_chunks] of var 1..boxes_along_x: x_start;

```

The variable `y_row[g]` denotes the row assigned to chunk g , while `x_start[g]` denotes the column where that chunk begins.

A first requirement is that every chunk must fit within the horizontal limits of the pallet. Since chunk g starts at column `x_start[g]` and has length `chunk_len[g]`, its last occupied column is `x_start[g] + chunk_len[g] - 1`. This position must not exceed the number of columns available on the pallet:

```

1 constraint forall(g in 1..n_chunks)(
2   x_start[g] + chunk_len[g] - 1 <= boxes_along_x
3 );

```

The second requirement is that two chunks must not occupy the same pallet cells. Each chunk can be seen as a horizontal rectangle of height 1: it starts at $(x_start[g], y_row[g])$, has width `chunk_len[g]`, and has height 1. Therefore, the non-overlap condition can be expressed directly using the global constraint `diffn`:

```

1 include "globals.mzn";    % for diffn
2
3 constraint diffn(x_start, y_row, chunk_len, [1 | g in 1..n_chunks]);

```

The constraint `diffn(x, y, dx, dy)` enforces that the rectangles defined by origins $(x[i], y[i])$ and dimensions $(dx[i], dy[i])$ do not overlap. In this model, all rectangles have height 1, so `diffn` ensures that chunks assigned to the same row occupy disjoint horizontal intervals.

Together with the chunking constraints, these placement constraints already define a valid pallet layout. The chunking constraints ensure that all `n_boxes` boxes are assigned to chunks, while the placement constraints ensure that those chunks fit inside the pallet and do not overlap.

However, one useful structural property is still only implicit: each pallet row must receive exactly `chunks_per_row` chunks. This follows from the parameter identity

$$n_chunks = boxes_along_y \cdot chunks_per_row,$$

combined with the non-overlap and capacity constraints. Nevertheless, leaving this property implicit makes the solver discover it through search.

To make this information explicit, the model adds a global cardinality constraint:

```

1 constraint global_cardinality(
2   y_row,
3   [r | r in 1..boxes_along_y],
4   [chunks_per_row | r in 1..boxes_along_y]
5 );

```

The constraint `global_cardinality(arr, vals, counts)` enforces that each value `vals[i]` appears in `arr` exactly `counts[i]` times. In this case, every pallet row from 1 to `boxes_along_y` must appear exactly `chunks_per_row` times in `y_row`. As a result, the solver knows from the beginning how many chunks must be assigned to each row. Once a row has received the required number of chunks, no further chunks can be placed there, which reduces unnecessary branching during search.

4 Search Strategy

4.1 Default search

The initial model relies on MiniZinc's default search:

```

1 solve satisfy;

```

With no annotation, the solver applies a generic variable- and value-selection heuristic (typically first-fail on the smallest domain, `indomain_min` on values) over all decision variables in the model. Given that `chunk_len`, `chunk_start`, `y_row`, `x_start`, and `plate` are tightly coupled through the geometric and collision constraints, the resulting search tree quickly becomes large.

4.2 Constructive branching

The refined model exploits a property of the problem: the chunks can be placed row by row, bottom to top, each one as long as the gripper allows and aligned to the left of the still-empty portion of its row. Under this plan, the row above each new chunk is already full, so the collision

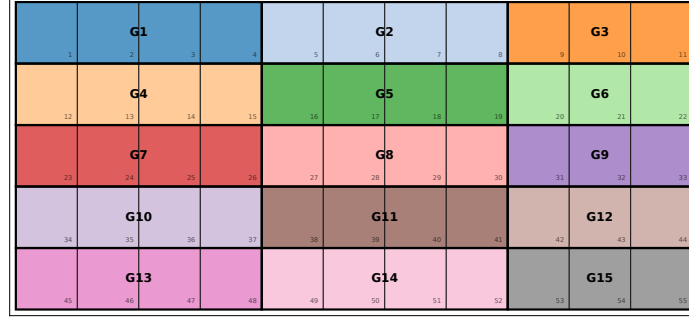


Figure 1: Example pallet layout generated for 70×70 boxes on an 800×1200 pallet. The resulting 11×17 grid contains 187 boxes. The grip size is set to three box widths. Chunks alternate 3/4/4 across each row.

rule forces the plate to open downward into the still-empty rows below, where no collision is possible. The plan is therefore feasible by construction, and a search strategy that visits the decision variables in this order finds a solution with essentially no backtracking.

The search must interleave four decisions per chunk ($y_row[g]$, $chunk_len[g]$, $x_start[g]$, $plate[g]$), but a single `int_search` annotation visits one array in index order and applies a single value-selection rule. To combine the four decisions, the model channels them into a single `branch` array, laid out in blocks of four per chunk:

```

1  par int: branch_ub = max([boxes_along_y, boxes_along_x, max_pkble_boxes]);
2
3  array[1..4*n_chunks] of var 0..branch_ub: branch;
4
5  constraint forall(g in 1..n_chunks)(
6    branch[4*g-3] = y_row[g] /\
7    branch[4*g-2] = max_pkble_boxes - chunk_len[g] /\
8    branch[4*g-1] = x_start[g] /\
9    branch[4*g]   = 1 - plate[g]
10 );
11
12 solve :: int_search(branch, input_order, indomain_min)
13         satisfy;

```

The four entries per chunk are ordered to reflect the constructive plan: row, length, column, plate. Since `indomain_min` always selects the smallest available value, two of the four decisions are channelled in complemented form so that the same rule produces the desired extremum. The entry `max_pkble_boxes - chunk_len[g]` is minimised when `chunk_len[g]` is maximised, yielding the longest pick the gripper allows; the entry `1 - plate[g]` is minimised when `plate[g] = 1`, selecting the downward plate. The two non-complemented entries directly produce the smallest row and the leftmost column.

The variable `chunk_start` does not appear in the `branch` array. The prefix-sum constraints introduced in Section 2 make it a deterministic function of the chunk lengths, so the search can leave it to propagation alone and avoid exploring redundant assignments of belt positions.

An example of the result of the chunking phase is shown in Figure 1.

5 Calling the Model from Python

Python is used to run the MiniZinc model on the benchmark instances. The script loads the model and selects the solver:

```
1 model = Model("model.mzn")
2 solver = Solver.lookup("gecode")
3 instance = Instance(solver, model)
```

It then sets the instance parameters:

```
1 instance["n_boxes"] = pallet_info["n_boxes"]
2 instance["boxes_x_dim"] = boxes_x_dim
3 instance["boxes_along_x"] = pallet_info["boxes_along_x"]
4 instance["boxes_along_y"] = pallet_info["boxes_along_y"]
5 instance["grip_size"] = grip_size
```

After the parameters are set, the script calls the solver:

```
1 result = instance.solve()
```

The output arrays `group_start`, `group_len`, `drop_order`, and `plate` are then read from the `result` map. They give the grouping of the boxes and the release schedule.

6 Benchmarking

The benchmark evaluates the MiniZinc model on every (pallet, pack) combination obtained from the pallet presets in Table 1 and the pack sizes in Table 2.

Pallet preset	Dimensions (mm)	Box size (mm)
Europallet	1200 × 800	50 × 50
Industrie	1200 × 1000	70 × 70
US 40×48	1219 × 1016	100 × 100
Half-pallet	800 × 600	120 × 80
Australian	1165 × 1165	150 × 150
		200 × 200

Table 1: Pallet presets used in the benchmark.

Table 2: Box sizes used in the benchmark.

For each combination, the model is tested with four solvers: `gecode`, `cp-sat`, `chuffed`, and `highs`. These solvers represent and solve the problem in different ways. `gecode` and `chuffed` mainly rely on constraint propagation, `cp-sat` uses a SAT-based representation, and `highs` uses a linear programming representation. Each run is executed with a timeout of 300 seconds. The benchmark considers three gripper sizes, $S \in \{1, 2, 3\} \cdot \text{boxes_x_dim}$, allowing us to compare how the gripper size affects grouping and scheduling performance.

6.1 Scaling by gripper size

The first comparison fixes the gripper multiplier and measures how the solve time changes with the total number of boxes. Figure 2 shows the resulting scaling curves for $k = 1$, $k = 2$, and $k = 3$. This analysis captures the effect of the gripper size on the proposed model. Larger grippers reduce the number of groups that must be scheduled: the average number of groups

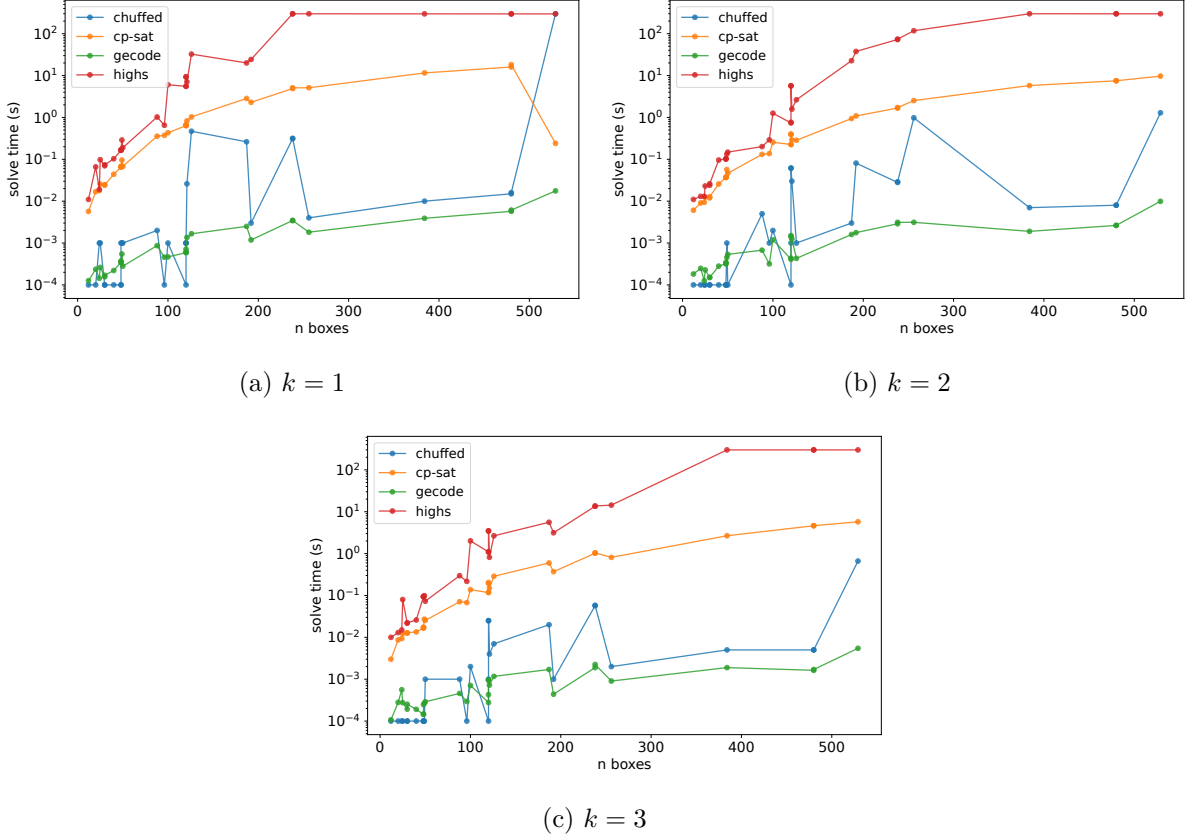


Figure 2: Runtime scaling for the three gripper multipliers $k \in \{1, 2, 3\}$. The y -axis uses a logarithmic scale.

decreases from about 75 for $k = 1$ to about 40 for $k = 3$. Therefore, the solve time generally decreases as the multiplier increases.

In particular, **gecode** remains in the lowest runtime range and shows the best scaling behaviour. **cp-sat** shows a clearer dependence on the number of boxes, but remains well below the timeout. **chuffed** is usually fast, although it times out on the densest instance for $k = 1$. **highs** is consistently the slowest solver.

6.2 Overall solver behaviour

After separating the gripper sizes, we aggregate all benchmark runs into a single scaling plot. Figure 3 reports the solve time as a function of the total number of boxes, considering all solvers and all tested instances. This view highlights the global trend across the benchmark and makes the relative behaviour of the solvers easier to compare.

Overall, the comparison confirms that the propagation-based solvers are the most suitable for this model. **gecode** gives the most stable performance and remains close to the bottom of the plot for almost all instances. **chuffed** is also competitive, but its performance is less regular. **cp-sat** shows a stronger dependence on the instance size, although it still remains far from the timeout in most runs. In contrast, **highs** performs worst and several runs approach or reach the timeout.

6.3 Per-pallet breakdown

Another useful comparison studies how the pallet preset influences solver runtime. Differences across presets reflect how **boxes_along_x** and **boxes_along_y** determine the number and

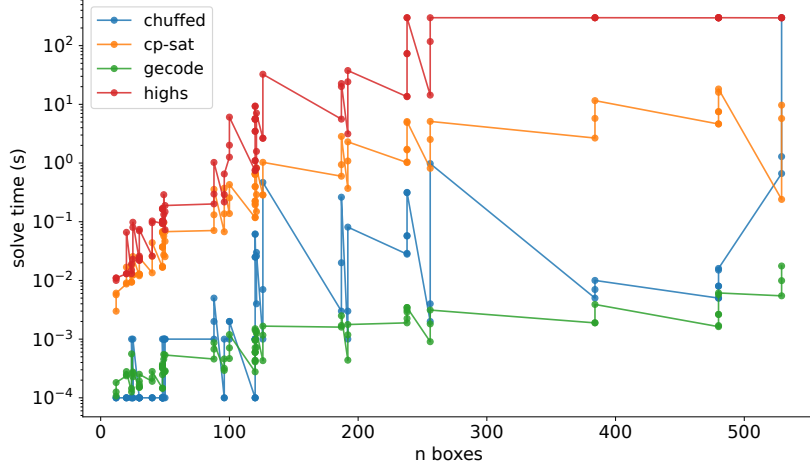


Figure 3: Aggregate runtime scaling over all benchmark instances and solvers. The y -axis uses a logarithmic scale.

arrangement of groups. Figure 4 shows the resulting breakdown.

The relative behaviour of the solvers remains mostly unchanged across pallets: **gecode** stays the most stable, **cp-sat** and **chuffed** remain competitive, and **highs** is usually the slowest. Larger presets, especially the Industrie, US, and Australian pallets, make the separation between solvers more visible because they include denser instances. The half-pallet instances are smaller, so the differences are less pronounced.

6.4 Flattening and execution overhead

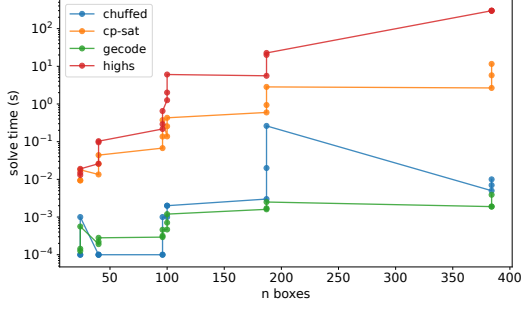
The last analysis compares the solver-reported time with the total runtime measured by the Python benchmark script. This comparison estimates the cost of the steps that occur before the actual solver search. The main one is MiniZinc flattening: in this phase, the high-level MiniZinc model and the instance data are translated into a lower-level FlatZinc representation, which is then passed to the selected backend solver. The remaining overhead includes Python instance construction, MiniZinc invocation, and process startup.

Figure 5 shows the resulting comparison. The diagonal line marks equal total runtime and solver-reported time. Points close to the diagonal correspond to runs where most of the time is spent inside the solver. Points above the diagonal instead indicate a larger overhead outside the solver; in these cases, flattening and execution costs are more relevant relative to the actual solving time.

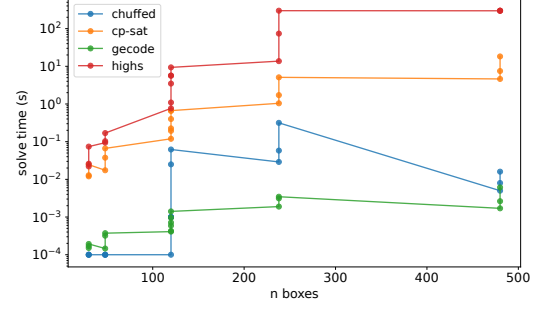
The plot shows that this effect is strongest for the fastest runs, especially for **gecode** and several **chuffed** executions. Their solver-reported time is very small, so even a modest overhead produces a visible separation from the diagonal. For slower runs, especially **highs** and the larger **cp-sat** instances, the points are closer to the diagonal. In those cases, the solving phase dominates the total runtime, and the relative overhead becomes less important.

7 Conclusions

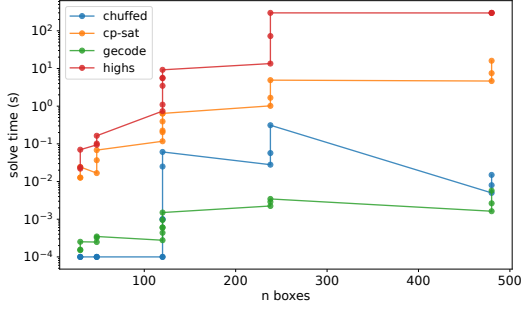
The project shows that the robot movement planning problem can be modeled compactly by separating grouping and scheduling decisions. The interval representation of groups, based on start index and length, avoids larger membership-based encodings while still capturing the gripper constraints. The release order is then handled through ordering variables and local neighborhood constraints.



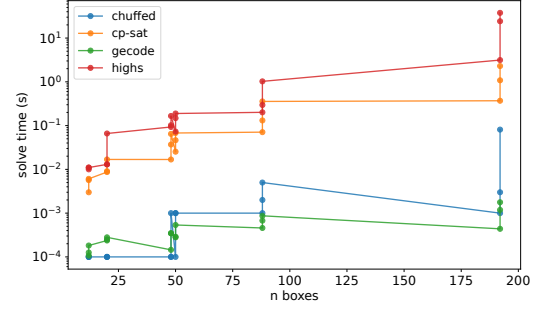
(a) Europallet



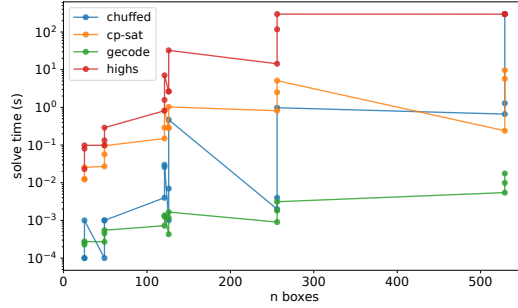
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 4: Runtime comparison across pallet presets. Each subplot aggregates all gripper multipliers for the corresponding preset. The y -axis uses a logarithmic scale.

The benchmark shows that the CP-oriented solvers are the most effective on the tested instances. `gecode` is the most stable solver, while `cp-sat` and `chuffed` remain competitive. In contrast, `highs` is consistently slower on this combinatorial formulation. Larger grippers reduce the number of groups and generally improve runtime, while dense layouts with many small boxes remain the hardest cases.

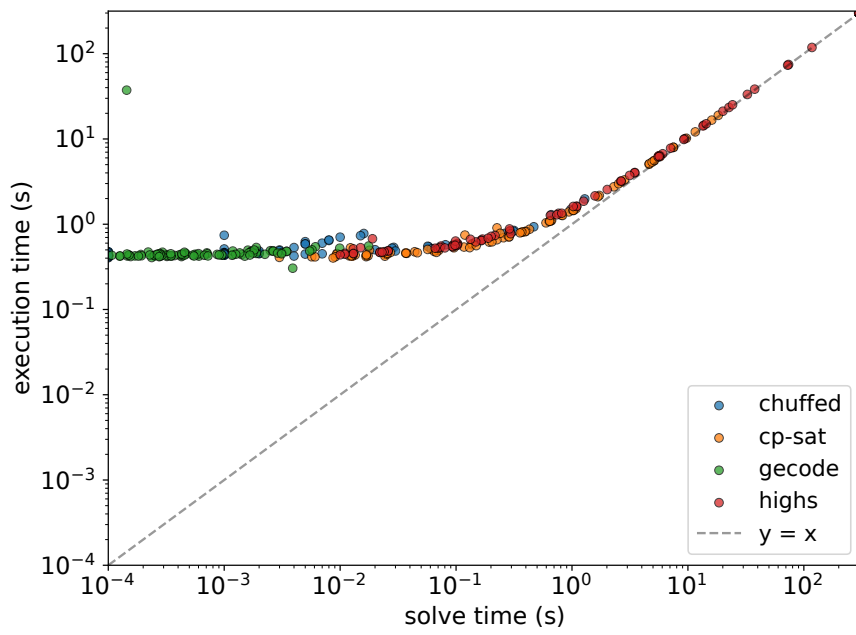


Figure 5: Execution time compared with solver-reported solve time. The diagonal marks equal values; points above it indicate additional overhead before or around the solver call. Both axes use a logarithmic scale.